AEGIS AGENT | BENCHMARKING

Performance, Reliability, Security and Evidence Methodology

Performance and Evidence Philosophy

This document defines a rigorous, reproducible evaluation framework for Aegis Agent and its separately implemented private intelligence backend. It distinguishes observed results from test design and does not present fabricated measurements.

Benchmarking is not a competition to maximize a single score. The engineering objective is a measured balance of task correctness, security preservation, token efficiency, retrieval quality, reliability, resource consumption and operational consistency.

Every reported outcome requires a clearly defined workload, environment, measurement procedure, repetition count, statistical summary and disclosure of limitations. Without executed measurements, the corresponding result remains unreported.

Evaluation Domains and System Boundaries

Treat the public Aegis host and private intelligence backend as separate measurement targets. The private backend is not yet connected to the public host. End-to-end integrated measurements are therefore not represented as existing results.

The host evaluation covers routing correctness, specialist workflow behavior, model usage, authorization integrity and task completion. The private backend evaluation covers structural understanding, evidence retrieval, context isolation, indexing and persistence.

Combined-system evaluation becomes meaningful only when a real integration exists. Until then, component-level results must not be aggregated into a fictional integrated score.

Measurement Integrity and Reproducibility

Each test case must have a fixed input, expected behavior, authorized data scope, acceptance condition and repeatable execution procedure. Test artifacts should include sanitized configuration, corpus fingerprints, harness configuration and machine-readable outcomes.

Measure warm and cold execution separately. Control model selection, hardware class, background load, corpus size, repository language composition and cache state. Never compare results from materially different conditions without qualification.

Separate development diagnostics from externally defensible evidence. A claim is reportable only when the underlying run can be reproduced and independently reviewed without exposing private implementation.

Benchmark Workload Taxonomy

Create representative workloads for source navigation, symbol dependency analysis, code explanation, cross-file reasoning, security review, multi-agent task routing, context assembly, incremental indexing and authorization rejection.

Include simple, moderate and difficult tasks, as well as ambiguous queries, incomplete source files, stale indexes, overlapping symbol names, long dependency chains and contradictory evidence.

Use both realistic engineering tasks and controlled synthetic cases. Synthetic workloads isolate causal factors; real repositories reveal behavior under practical structural complexity.

Task Correctness and Outcome Quality

Evaluate whether the system identifies the right agent, retrieves the right source evidence and produces conclusions supported by that evidence. Accuracy must be assessed against an independently prepared answer key or adjudicated reference.

Track exact-match correctness where appropriate, evidence-supported answer rate, unsupported assertion rate, omission rate and task-specific acceptance. Distinguish partial correctness from unsupported certainty.

For multi-agent workflows, evaluate agreement, conflict detection, handoff fidelity and whether an agent mistakenly presents another specialist’s inference as verified fact.

AST Structural Extraction Quality

Measure parser coverage by language and construct, symbol detection precision and recall, source-span accuracy, reference resolution and graph edge correctness. Include syntax errors, generated files and language features with known parser limitations.

Use labeled code fixtures with ground-truth definitions, imports, calls and dependencies. Report results by language and difficulty rather than hiding weak cases inside a global average.

An inferred graph edge must not count as a verified structural fact. Evaluate parser-derived edges and probabilistic associations separately.

Semantic Graph Retrieval Quality

Construct task queries whose relevant source symbols and dependency neighborhoods are known in advance. Evaluate Recall at K, Precision at K, mean reciprocal rank and nDCG where ranking is meaningful.

Measure graph-hop expansion usefulness, irrelevant-edge amplification, source revision consistency and sensitivity to repository scale. Inspect cases where relevant evidence exists but is not retrieved.

Assess the tradeoff between retrieval breadth and token budget. Higher recall is not useful if excessive unrelated context degrades reasoning or increases disclosure risk.

Task-Scoped Retrieval Isolation

Test that retrieval returns only evidence authorized for the active principal, task, agent and repository. Construct negative controls with similar queries across different scopes and deliberate cache reuse attempts.

Measure unauthorized retrieval count, scope-policy violation rate, revocation handling and cache contamination. The acceptance expectation for unauthorized evidence disclosure is zero in the defined test corpus.

Report the test coverage and attack conditions. Passing a finite isolation suite is evidence for those conditions, not proof of immunity against every conceivable attack.

Context Purity and Contamination

Create labeled corpora with relevant evidence, distractors, conflicting documents, prompt-injection payloads and cross-task material. Assess whether the assembled context remains scoped, source-attributed and free of unauthorized passages.

Measure context relevance ratio, duplicate passage rate, missing critical evidence, provenance completeness and contamination incidents. Inspect whether truncation removes an essential caveat or dependency.

An integrity digest demonstrates snapshot consistency, not factual correctness. Evaluate source attribution and semantic accuracy independently.

Token Efficiency and Information Density

Count prompt, completion and total tokens per successful task using actual provider or tokenizer accounting. Report medians, tail percentiles, success-normalized token consumption and variation across task complexity.

Compare equivalent tasks under controlled retrieval strategies: unfiltered context, lexical-only selection and structural task-scoped evidence when those configurations genuinely exist and are available for testing.

Measure tokens per correct answer and tokens per fully supported answer. A reduction in tokens is beneficial only if correctness, provenance and security acceptance remain within defined limits.

Latency and Throughput

Measure end-to-end task latency for the host and stage-level latency for private indexing, authorization, graph traversal, ranking, context assembly and model inference. Use monotonic clocks and separate cold starts.

Report median, p90, p95 and p99 latency when sample size supports meaningful tail estimation. Throughput is expressed as completed valid tasks per unit time at stated concurrency and workload composition.

Account for queueing and backpressure. A fast rejected task must not be counted as successful throughput. Include cancellation, timeout and resource saturation behavior.

Resource Consumption and Local Operation

Observe peak and steady-state memory, CPU time, disk utilization, index storage growth and I/O activity under defined workloads. Attribute resource use to ingestion, graph storage, retrieval, model execution and orchestration where possible.

For local-first operation, include offline scenarios and compare idle footprint with active processing. Note whether a model runtime or external provider is included in resource totals.

Cost reporting should distinguish model tokens, local compute time and operational overhead. Do not translate token counts into monetary claims without a documented pricing or infrastructure model.

SQLite Concurrency Measurement

Benchmark read-heavy, write-heavy and mixed workloads using explicit connection counts, transaction sizes, queue policies, index sizes and WAL configuration. SQLite supports overlapping reads and writes in WAL mode but retains a single-writer limitation.

Measure transaction success, SQLITE_BUSY and SQLITE_LOCKED events, writer queue delay, tail write latency, checkpoint pressure and integrity after interruption. Include concurrent index ingestion and reader snapshot tests.

Correctness is primary: published graph revisions must remain atomic and readers must never observe a partially committed semantic index. Report contention transparently rather than claiming universally lock-free writes.

Reliability and Fault Injection

Inject controlled failures such as service unavailability, storage exhaustion, parser crashes, malformed input, interrupted indexing, slow reads, timeouts and cancelled requests. Measure recovery behavior and consistency of completed outputs.

Track successful task completion, data integrity, bounded error response and restoration of a coherent index. A partial or rejected operation is not automatically a reliability failure when denial is the correct secure outcome.

The benchmark reports behavior under failure; it does not prescribe an integration implementation plan or assert that unconnected systems already demonstrate combined resilience.

Agentic Security Evaluation

Test prompt injection in code comments, README files, retrieved documents and tool responses. Attempt unauthorized tool invocation, tenant confusion, stale-context reuse, privilege escalation through handoffs and leakage through generated answers.

Report attack success under precisely defined attacker control, policy bypass incidence, false rejection of legitimate tasks and evidence of data exposure. Security outcomes require manual review of high-impact cases.

The benchmark must not publish secrets, live exploit credentials or proprietary private data. Sanitized attack fixtures and outcome categories can support review without disclosing sensitive implementation.

Multi-Agent Workflow Assessment

Evaluate specialist routing accuracy, handoff correctness, duplicated reasoning, unsupported consensus and role-appropriate context allocation. Compare individual and collaborative task outcomes on the same reference tasks.

Track the number of agent invocations, evidence items exchanged, context tokens per specialist and final answer support. More agent calls are not inherently better.

Distinguish a sequential six-agent workflow from adaptive collaboration. The public host must be evaluated according to behavior actually implemented in the tested revision.

Statistical Treatment and Reporting

Use repeated runs and describe variability. For rates, report numerator and denominator; for continuous measurements, report distribution summaries and suitable uncertainty intervals when statistically justified.

Prefer paired comparisons on identical tasks when evaluating retrieval strategies. Define exclusion rules before execution and retain failed runs in the analysis unless a documented measurement fault justifies exclusion.

Do not use a single composite score to conceal regressions in security or correctness. Report each critical dimension separately and explain tradeoffs.

Acceptance Criteria and Evidence Gates

Each benchmark suite must define a measurable pass condition before execution. Security isolation, action authorization and data integrity are hard constraints; efficiency improvements cannot compensate for violations.

Correctness, retrieval quality, token efficiency, latency and resource usage should have workload-specific thresholds selected from the actual engineering requirements. No universal target is invented without empirical basis.

Results should be classified as executed and supported, executed with limitations, or not yet measured. The report must not convert a designed test into a passed test.

Black-Box Assessment of Private Intelligence

The private backend can be evaluated through an authorized, isolated interface using fixed requests and validated outputs. The test harness needs access to observable behavior, resource measurements and sanitized evidence—not the proprietary source repository.

Use ground-truth code fixtures, scoped retrieval requests, malformed inputs, revocation cases, concurrent database workloads and reproducible index revisions. Verify both positive results and deliberate denial paths.

Black-box evidence can establish behavior under tested conditions while preserving intellectual property. It cannot establish unseen internal controls solely from favorable output.

Publication and Disclosure Discipline

Publish only results backed by executed runs, reproducible methodology and an appropriate disclosure review. Where data cannot be shared, describe the corpus characteristics, test conditions and verification limits without implying public reproducibility that is not available.

Do not place private code, credentials, sensitive graph data or internal deployment topology into a public benchmarking artifact. Use anonymized evidence and disclose material constraints.

The document is an independent benchmarking specification. It does not serve as an implementation directive, an integration roadmap or a substitute for the separate Aegis engineering architecture.

Final Evaluation Standard

A credible Aegis benchmark demonstrates what was measured, how it was measured, under which conditions and with what uncertainty. It does not substitute architectural ambition for observed outcomes.

The evaluation framework prioritizes source-grounded correctness, isolation, security, token discipline, operational reliability and clear boundaries between the public host and private backend.

No end-to-end integrated benchmark is claimed while the private backend remains unconnected to Aegis. Any future integrated result must be independently measured under its own stated conditions.